

## گزارش سوال ۳ عملی - تمرین ۶ - سیستم عامل

---

ابتدا به طور خلاصه نگاهی به پیاده‌سازی این تمرین می‌پردازیم، سپس دستورات جدید را امتحان کرده و از عملکرد درست آن‌ها اطمینان حاصل می‌کنیم. ابتدا در فایل fs.h استراکتهای زیر را تعریف می‌کنیم:

```
// ----- HW6 -----
#define MAX_NAME 32
#define MAX_USERS 16
#define MAX_GROUPS 16

// Permission bits (standard Unix-style)
// Read: 4, Write: 2, Execute: 1
// Example: 755 (Owner: rwx, Group: r-x, Others: r-x)
typedef struct {
    uint32_t owner_id;
    uint32_t group_id;
    uint32_t mode; // e.g., 755
} permission_t;

typedef struct {
    uint32_t id;
    char name[MAX_NAME];
    uint32_t group_ids[MAX_GROUPS]; // Groups this user belongs to
    uint32_t group_count;
} user_t;

typedef struct {
    uint32_t id;
    char name[MAX_NAME];
} group_t;
```

سپس توابع مورد نیاز را در fs.c پیاده‌سازی می‌کنیم:

```
// ----- HW6 -----  
// ----- Users & Permissions -----  
user_t g_users[MAX_USERS];  
group_t g_groups[MAX_GROUPS];  
uint32_t g_user_count = 0;  
uint32_t g_group_count = 0;  
uint32_t g_current_user_id = 0; // Default to Root  
  
> void init_root() { ...  
|  
// Command Implementations  
> void user_add(const char *username) { ...  
  
> void usermod_ag(const char *groupname, const char *username) { ...  
  
> bool check_permission(permission_t *p, int required) { ...  
  
> void chmod_file(uint32_t mode) { ...  
  
> int get_uid_by_name(const char *name, superblock_t *sb) { ...  
  
> int get_gid_by_name(const char *name, superblock_t *sb) { ...  
  
> void chown_file(const char *user_name, const char *group_name) { ...  
  
> void getfacl_file() { ...  
  
  
> bool login_user(const char *username) { ...  
  
> void chgrp_file(const char *group_name) { ...  
  
> uint32_t get_primary_gid(uint32_t uid) { ...  
  
> void sync_users_from_disk() { ...  
  
> void user_del(const char *username) { ...  
  
> void group_add(const char *groupname) { ...  
  
> void group_del(const char *groupname) { ...  
  
> void list_users_and_groups() { ...
```

نهایتاً دستورات جدید را به cli.c اضافه می‌کنیم:

```
// ----- HW6 -----  
// ----- Users & Permissions -----  
else if (strcmp(tokens[0], "useradd") == 0) { ...  
else if (strcmp(tokens[0], "login") == 0) { ...  
else if (strcmp(tokens[0], "userdel") == 0) { ...  
else if (strcmp(tokens[0], "groupadd") == 0) { ...  
else if (strcmp(tokens[0], "groupdel") == 0) { ...  
else if (strcmp(tokens[0], "usermod") == 0) { ...  
else if (strcmp(tokens[0], "chmod") == 0) { ...  
else if (strcmp(tokens[0], "chown") == 0) { ...  
else if (strcmp(tokens[0], "chgrp") == 0) { ...  
else if (strcmp(tokens[0], "getfacl") == 0) { ...  
else if (strcmp(tokens[0], "users") == 0) { ...
```

همچنین لازم بود تغییراتی در توابع قبلی ایجاد کنیم تا ملاحظات امنیتی اضافه شده را رعایت کنند. برای مثال هنگام read

و write فایل‌ها، ابتدا دسترسی کاربر را بررسی می‌کنیم:

```
// ----- read/write -----  
void write_file(uint32_t pos, uint32_t nbytes) {  
    if (g_open_file_offset == 0) { printf("No file open.\n"); return; }  
  
    file_entry_t ent;  
    load_file_entry(g_open_file_offset, &ent);  
  
    // 2 is the bit for WRITE  
    if (!check_permission(&ent.perm, 2)) {  
        printf("Permission denied: cannot write.\n");  
        return;  
    }  
}
```

در ادامه به بررسی عملکرد دستورات می‌پردازیم:

1. useradd <username>
2. groupadd <groupname>
3. usermod -aG <group> <username>
4. users

همانطور که مشاهده می‌کنید، علاوه بر دستورات خواسته شده، دستور users را اضافه کردیم که تمام user ها و group های موجود را نشان می‌دهد.

```
PS Microsoft.PowerShell.Core\FileSystem:
FS CLI - type 'help' for commands.
myfs> init fs2
Created new filesystem 'fs2'
myfs> useradd user1
User 'user1' added successfully.
myfs> useradd user2
User 'user2' added successfully.
myfs> groupadd group1
Group 'group1' added.
myfs> groupadd group2
Group 'group2' added.
myfs> users

Users:
  root (uid=0): root
  user1 (uid=1): (no groups)
  user2 (uid=2): (no groups)

Groups:
  root (gid=0)
  group1 (gid=1)
  group2 (gid=2)

myfs> usermod -aG group1 user1
User user1 added to group group1.
myfs> usermod -aG group2 user2
User user2 added to group group2.
myfs> users

Users:
  root (uid=0): root
  user1 (uid=1): group1
  user2 (uid=2): group2

Groups:
  root (gid=0)
  group1 (gid=1)
  group2 (gid=2)
```

5. userdel <username>

6. groupdel <groupname>

7. login <username>

همچنین دستور login را اضافه کردیم که اجازه می‌دهد به عنوان یک user خاص فعالیت کنیم. مشاهده می‌کنید که برخی دستورات (مانند افزودن و یا حذف کاربر و گروه و ...) فقط توسط کاربر root قابل اجرا هستند.

8. getfacl

با این دستور اطلاعات دسترسی و مالکیت فایل باز را مشاهده می‌کنیم. می‌بینیم که فایلی که توسط user1 ایجاد شده توسط user2 قابل write نیست و فقط می‌تواند آن را بخواند.

```
Users:
  root (uid=0): root
  user1 (uid=1): group1
  user2 (uid=2): group2

Groups:
  root (gid=0)
  group1 (gid=1)
  group2 (gid=2)

myfs> userdel user1
User 'user1' deleted.
myfs> groupdel group1
Group 'group1' deleted.
myfs> users

Users:
  root (uid=0): root
  user2 (uid=2): group2

Groups:
  root (gid=0)
  group2 (gid=2)

myfs> login user2
Logged in as user2 (UID: 2)
myfs> useradd user3
Permission denied: Only root can add users.
myfs> groupdel group2
Permission denied: only root can delete groups.
myfs> █
```

```
myfs> login user1
Logged in as user1 (UID: 1)
myfs> open file1 1
File created and opened: file1
myfs> write 0 5
12345
Wrote 5 bytes.
myfs> myfs> getfacl

# file: file1
owner: user1 (id: 1)
group: group1 (id: 1)
user: rw-
group: r--
other: r--
myfs> login user2
Logged in as user2 (UID: 2)
myfs> write 0 5
Permission denied: cannot write.
myfs> read 0 100
12345
myfs> █
```

## 9. chmod <mod>

مشاهده می‌شود که این دستور توسط مالک فایل یا root قابل اجرا است. پس از افزایش دسترسی به 777، کاربران دیگر نیز می‌توانند درون فایل بنویسند.

```
myfs> login user2
Logged in as user2 (UID: 2)
myfs> write 0 5
Permission denied: cannot write.
myfs> read 0 100
12345
myfs> chmod 777
Permission denied.
myfs> login user1
Logged in as user1 (UID: 1)
myfs> chmod777
Unknown command: chmod777
Type 'help' for command list.
myfs> chmod 777
Permissions updated to 777
myfs> login user2
Logged in as user2 (UID: 2)
myfs> write 5 5
user2
Wrote 5 bytes.
myfs> myfs> read 0 100
12345user2
myfs>
```

## 10. chown <user>:<group>

با این دستور کاربر و گروه مالکیتی فایل را تغییر می‌دهیم.

```
myfs> getfacl

# file: file1
owner: user2 (id: 2)
group: group2 (id: 2)
user: rwx
group: rwx
other: rwx
myfs> chgrp group1
Group changed to group1
myfs> getfacl

# file: file1
owner: user2 (id: 2)
group: group1 (id: 1)
user: rwx
group: rwx
other: rwx
myfs>
```

```
myfs> login root
Logged in as root (UID: 0)
myfs> chown user2:group2
Ownership updated to user2:group2
myfs> getfacl

# file: file1
owner: user2 (id: 2)
group: group2 (id: 2)
user: rwx
group: rwx
other: rwx
myfs>
```

## 11. chgrp <group>

با این دستور تنها گروه مالکیتی فایل را تغییر می‌دهیم.